



LYRA RECORDER

Ficha técnica · Versión 1.0

Resumen

Lyra Recorder es una aplicación profesional de captura de audio para Android, con calibración por sesión, medidores estilo hardware, limitador de recorte duro y detección adaptativa de silencio. La cadena de señal está implementada en código Kotlin en coma flotante sobre AudioRecord y MediaCodec para captura de baja latencia y codificación de alta calidad.

Motor de audio

API de captura	Android AudioRecord (Kotlin Native)
API de codificación	MediaCodec + MediaMuxer (AAC)
Frecuencia de muestreo	44100 Hz
Profundidad de bits	16-bit PCM
Canales	Mono
Procesamiento interno	32-bit floating point
Tamaño del búfer	AudioRecord.getMinBufferSize (≥ 4096)

Cadena de señal

Orden de procesamiento, desde la entrada del micrófono hasta la salida a archivo:

1. Conversión	PCM 16-bit $\rightarrow$ float [-1.0, +1.0]
2. HPF (opcional)	IIR 1-pole, $\alpha \approx 0.989$ (~80 Hz)
3. HPF de detección	AUTO PAUSE only
4. RMS (detección)	Silence/voice threshold compare
5. AUTO PAUSE	Sample counter vs thresholds
6. GAIN	User multiplier (knob)
7. LIMITER (opcional)	Hard clip ± 0.90 (~-0.9 dBFS)
8. Cuantización	Float $\rightarrow$ 16-bit PCM
9. Salida	WAV / AAC + monitor buffer
10. Análisis	FFT + waveform + VU RMS

Calibración por sesión

Un test de dos fases mide el micrófono y el entorno antes de cada grabación calibrada. A partir de estas medidas la app deriva la ganancia de los medidores, los umbrales de auto-pausa y la viabilidad de AUTO PAUSE.

Duración fase 1	5 segundos (silencio)
Duración fase 2	5 segundos (voz)
Salida: noiseFloor	RMS medio durante la fase de silencio
Salida: voiceLevel	RMS medio durante la fase de voz

Derivado: meterGain	$(0.5 / (\text{voiceLevel} \times 3.0)), \text{ clamp } [0.1, 4.0]$
Derivado: pauseThreshold	$(\text{noiseFloor} \times 1.8), \text{ clamp } [0.002, 0.5]$
Derivado: resumeThreshold	$(\text{noiseFloor} \times 3.5), \text{ clamp } [0.004, 0.8]$
Viabilidad de auto-pausa	$\text{voiceLevel} \geq \text{noiseFloor} \times 4 \text{ AND } \text{noiseFloor} < 0.05$
Umbral de detección de voz	$\text{voiceLevel} \geq 0.015 \text{ AND } \geq \text{noiseFloor} \times 2.0$

Medición

Rango VU	-20 dB → +3 dB
Respuesta VU	Asimétrica (ataque rápido 0,55, decaimiento lento 0,10)
Techo VU (limiter desactivado)	+3 dBFS
Techo VU (limiter activado)	-1 dBFS (duro, clamp en dos etapas)
Umbral LED de clip	≥ -3 dBFS rojo, ≥ -7 dBFS ámbar
Bandas del espectro	64 (segmentos RMS en dominio temporal)
Resolución de forma de onda	120 muestras / frame
Frecuencia de actualización	~60 Hz (Flutter Ticker)

Formatos de salida

WAV	PCM 16 bits, 44,1 kHz, mono, sin comprimir
AAC	MPEG-4 AAC-LC, 256 kbit/s, 44,1 kHz, mono, contenedor .m4a
Nomenclatura de archivos	lyra_.
Archivo de marcadores	Mismo nombre, sidecar .json

Funcionalidades

Modos de grabación	Directo (crudo) · Calibrado (funciones completas)
Bloqueo por calibración	HPF, LIMITER, AUTO PAUSE requieren perfil activo
Marcadores	Por grabación, con nota de texto opcional (≤ 50 caracteres)
Modos de auto-pausa	OFF · 1 s de silencio · 2 s de silencio
Modo monitor	Bucle de baja latencia vía AudioRecord + AudioTrack
Sistema de layout	FixedCanvas (FittedBox + referencia 423,5×945,9 dp)
Recorte de pantalla	LAYOUT_IN_DISPLAY_CUTOUT_MODE_SHORT_EDGES

Requisitos de plataforma

Sistema operativo	Android 7.0 (API 24) o superior
Arquitectura	arm64-v8a, armeabi-v7a, x86_64
Permisos	RECORD_AUDIO, FOREGROUND_SERVICE, WRITE_EXTERNAL_STORAGE
Tamaño aproximado del APK	< 30 MB
Requisito de almacenamiento	~10 MB por minuto (WAV) / ~2 MB por minuto (AAC)

Implementación

Capa UI	Flutter / Dart
Capa nativa	Kotlin (Android)
Nombre del canal	com.example.lyra_recorder/recorder
Canal de eventos	com.example.lyra_recorder/audio_stream
Hilo de audio	Hilo Kotlin dedicado, prioridad de tiempo real